
INSTITUTO TECNOLÓGICO DE MAZATLÁN

Tecnológico Nacional de México

Reporte Técnico de Prácticas de Automatización y Monitoreo Ambiental con Arduino UNO R4 WiFi, Sensor DHT11 e Integración IoT

Alumna:	Díaz Camacho Karla Mariel
---------	---------------------------

Matrícula:	251000165
------------	-----------

Materia:	Desarrollo Sustentable
----------	------------------------

Profesor:	Barrón Hernández Miguel Ángel
-----------	-------------------------------

Fecha:	26 de septiembre de 2026
--------	--------------------------

Materia: Desarrollo Sustentable con Automatización

1. Resumen ejecutivo / Introducción

El presente reporte documenta el desarrollo secuencial de cuatro prácticas de automatización e Internet de las Cosas (IoT), realizadas con la placa Arduino UNO R4 WiFi como plataforma de control y el sensor DHT11 como elemento de sensado ambiental. El propósito de estas prácticas fue avanzar progresivamente desde el control básico de un actuador a través de una red WiFi local, hasta la integración de un sistema completo de monitoreo y alerta capaz de comunicarse con servicios en la nube mediante Make.com y notificar al usuario en tiempo real a través de Telegram.

Esta secuencia de prácticas permite observar la evolución de un mismo proyecto: se inicia con el manejo de peticiones HTTP y el control remoto de un LED, continúa con la lectura confiable de variables ambientales (temperatura y humedad), incorpora una lógica de decisión local basada en un umbral de temperatura, y culmina con la conexión del dispositivo físico a un flujo de automatización en la nube que traduce una condición ambiental en una notificación automática. En conjunto, este proceso ejemplifica cómo la automatización y el monitoreo ambiental, aplicados de forma escalonada, contribuyen a los objetivos del Desarrollo Sustentable al optimizar el uso de recursos como la energía eléctrica mediante decisiones basadas en datos reales del entorno.

2. Desarrollo metodológico y prácticas previas (secuencia del proyecto)

A continuación se describe, en orden cronológico, la lógica técnica y la implementación de cada una de las cuatro prácticas que conforman la secuencia del proyecto.

2.1 Práctica 1: Control de LED vía WiFi

Esta primera práctica tuvo como finalidad establecer el control remoto de un LED conectado a la placa Arduino UNO R4 WiFi, utilizando como interfaz un navegador web desde cualquier dispositivo conectado a la misma red inalámbrica.

Servidor web local (WiFiServer)

El programa establece la conexión a la red WiFi mediante `WiFi.begin(ssid, password)` y, una vez conectado, inicializa un servidor web local en el puerto 80 a través de la instrucción `WiFiServer server(80)`. Este servidor permanece a la escucha de conexiones entrantes dentro del ciclo principal del programa, revisando en cada iteración si algún cliente (por ejemplo, un navegador) solicita una conexión mediante `server.available()`. La dirección IP asignada por el router se imprime en el Monitor Serial para que el usuario pueda acceder a ella desde su celular o computadora.

Procesamiento de peticiones HTTP GET

Cuando un cliente se conecta, el Arduino lee la primera línea de la petición HTTP y determina la acción solicitada revisando si la cadena recibida contiene las rutas `/led/on` o `/led/off`. Dependiendo del resultado, se actualiza una variable booleana de estado (`ledState`) y se escribe el nivel lógico correspondiente en el pin del LED mediante `digitalWrite()`.

```
if (request.indexOf("GET /led/on") >= 0) {
```

```

        ledState = true;
        digitalWrite(ledPin, HIGH);
    } else if (request.indexOf("GET /led/off") >= 0) {
        ledState = false;
        digitalWrite(ledPin, LOW);
    }
}

```

Tras procesar la petición, el Arduino responde generando dinámicamente una página HTML que muestra el estado actual del LED ("ENCENDIDO" o "APAGADO") y presenta dos botones enlazados a las rutas mencionadas, permitiendo alternar el estado del actuador con un simple clic desde el navegador.

2.2 Práctica 2: Lectura de temperatura y humedad

La segunda práctica se centró en la lectura confiable de las variables ambientales de temperatura y humedad relativa mediante el sensor DHT11, sentando la base de sensado que se reutilizaría en las prácticas posteriores.

Funcionamiento del sensor DHT11 e intervalo mínimo de lectura

El sensor DHT11 se conecta al pin digital 2 de la placa y se inicializa mediante la librería `DHT.h`, declarando el objeto correspondiente con `DHT dht(DHTPIN, DHTTYPE)`. Debido a las características del sensor, es indispensable respetar un intervalo mínimo de 2000 milisegundos entre lecturas consecutivas, lo cual se implementa mediante la instrucción de `delay(2000)` al inicio del ciclo principal; de no cumplirse este tiempo, el sensor puede entregar lecturas erróneas o nulas.

Validación de datos y salida por Monitor Serial

Las lecturas de humedad y temperatura se obtienen mediante `dht.readHumidity()` y `dht.readTemperature()`, esta última expresada en grados Celsius por defecto. Antes de utilizar los valores obtenidos, el programa valida que ambas lecturas sean numéricamente válidas mediante la función `isnan()`, descartando el ciclo en curso si se detecta un error de lectura del sensor.

```

    if (isnan(humedad) || isnan(temperatura)) {
        Serial.println("Error al leer el sensor DHT11");
        return;
    }

```

Una vez validados los datos, ambos valores se imprimen en el Monitor Serial configurado a 9600 baudios, mostrando la humedad relativa en porcentaje y la temperatura en grados Celsius, lo que permitió comprobar visualmente la estabilidad y consistencia de las lecturas del sensor.

2.3 Práctica 3: Control local de LED por umbral de temperatura

La tercera práctica incorporó una primera capa de lógica de decisión, tomando como base la lectura del sensor DHT11 de la práctica anterior para activar un actuador local (el LED) en función de un umbral de temperatura predefinido.

Configuración del umbral de temperatura

El umbral se define como una constante de tipo flotante al inicio del programa, `const float UMBRAL_TEMP = 30.0;`, estableciendo el valor de referencia de 30.0 °C a partir del cual se considera que la temperatura del entorno es elevada.

Lógica condicional de encendido y apagado

En cada ciclo de lectura, una vez validado el dato de temperatura, el programa evalúa mediante una estructura `if / else` si el valor medido es mayor o igual al umbral establecido. Si la condición se cumple, se enciende el LED y se imprime un mensaje indicativo en el Monitor Serial; en caso contrario, el LED permanece o se coloca en estado apagado.

```
if (temperatura >= UMBRAL_TEMP) {  
    digitalWrite(ledPin, HIGH);  
    Serial.println("LED ENCENDIDO (temperatura alta)");  
} else {  
    digitalWrite(ledPin, LOW);  
    Serial.println("LED apagado");  
}
```

Esta práctica representó el primer sistema de respuesta automática del proyecto, en el cual la placa Arduino toma una decisión de manera autónoma a partir de una variable ambiental, sin depender todavía de ningún servicio externo.

2.4 Práctica 4: Integración avanzada con IoT, Webhooks y Telegram

La cuarta y última práctica integró todos los elementos desarrollados previamente en un sistema completo de monitoreo y alerta, conectando la placa Arduino con un escenario de automatización en Make.com y un bot de Telegram, manteniendo el umbral de 30.0 °C como criterio de activación.

Servidor web dinámico de visualización

Además de la lógica de sensado y decisión, el Arduino sirve una página web propia, accesible desde cualquier dispositivo conectado a la misma red WiFi, que muestra en tiempo real la temperatura, la humedad y el estado del LED. La página incorpora una metaetiqueta de actualización automática cada 5 segundos, permitiendo un monitoreo continuo sin necesidad de recargar manualmente.

Envío de datos a Make.com mediante HTTPS y JSON

Cuando la temperatura alcanza o supera el umbral configurado, el Arduino establece una conexión segura hacia el servicio de Make.com utilizando `WiFiSSLClient`, y envía una petición HTTPS de tipo POST cuyo cuerpo contiene el dato de temperatura en formato JSON.

```
String jsonPayload = "{\"temperatura\":\" + String(temp) + "\"}";  
client.println("POST " + String(webhookPath) + " HTTP/1.1");  
client.println("Host: " + String(host));  
client.println("Content-Type: application/json");  
client.println("Content-Length: " + String(jsonPayload.length()));
```

Del lado de Make.com, un módulo de tipo Webhook personalizado recibe esta petición y detecta el campo "temperatura" contenido en el JSON, dejándolo disponible para ser utilizado por el resto del escenario.

Disparo por cambio de estado

Para evitar el envío de mensajes duplicados mientras la temperatura se mantiene elevada, el programa conserva una variable de estado (`estadoCaliente`) y solo genera la petición hacia Make cuando detecta un cambio de estado, es decir, la primera vez que la temperatura cruza el umbral desde un estado normal hacia uno caliente. Una nueva alerta solo vuelve a enviarse si la temperatura primero desciende por debajo del umbral y posteriormente lo supera de nuevo.

Módulo de Telegram Bot y mensaje de alerta

Una vez que el Webhook recibe el dato, el escenario ejecuta de manera automática un módulo de Telegram Bot configurado para enviar un mensaje de texto al chat previamente autorizado, insertando dentro del mensaje el valor real de temperatura recibido: “El cuarto está muy caliente ({{temperatura}}). Enciende el aire acondicionado.” De esta manera, el sistema completa el ciclo completo de automatización: sensado físico, decisión local, transmisión de datos a la nube y notificación inmediata al usuario a través de un canal de mensajería.

3. Preguntas de reflexión

Pregunta 1: ¿Cómo ayuda el uso de sensores de temperatura y automatización a cuidar el medio ambiente y ahorrar energía?

El uso de sensores de temperatura permite que dispositivos como el aire acondicionado se enciendan únicamente cuando realmente son necesarios, en lugar de mantenerse funcionando de manera constante. Esto reduce el consumo innecesario de energía eléctrica y, en consecuencia, disminuye el impacto ambiental asociado a su generación, favoreciendo un uso más racional y sustentable de los recursos energéticos disponibles.

Pregunta 2: ¿Por qué es mejor enviar la alerta a Telegram solo cuando la temperatura sube de 30 °C en lugar de enviarla todo el tiempo?

Enviar la alerta únicamente cuando la temperatura supera el umbral evita saturar al usuario con mensajes innecesarios y repetitivos, permitiendo que la notificación conserve su valor de urgencia. Además, este comportamiento reduce el número de peticiones enviadas a Make.com, optimizando el consumo de operaciones del escenario y evitando un uso excesivo de la conexión a internet del dispositivo.

Pregunta 3: ¿Qué ventaja tiene recibir una notificación en el celular por Telegram en comparación con solo ver encendido el LED en la tarjeta Arduino?

El LED encendido solo puede observarse si la persona se encuentra físicamente cerca de la tarjeta Arduino, mientras que la notificación por Telegram llega directamente al celular del usuario sin importar en qué parte del lugar se encuentre. Esto permite una respuesta más oportuna ante la condición de temperatura elevada, ya que la persona puede enterarse de inmediato y actuar en consecuencia, aun cuando no esté observando el dispositivo en ese momento.